

NoteHub要件定義書

システム概要

概要

項目	内容
システム名	NoteHub
システム概要	NoteHubは複数ユーザーが同一ドキュメントをリアルタイムで共同編集できるオンラインエディタ
利用対象	共同作業を行う利用者
開発目的	チーム全体でリアルタイムに共同編集を行い、情報共有を効率化する

解決したい課題

チーム単位での作業をリアルタイムで可視化しながら、共同作業できる環境を提供する

ユーザーへの解決したい課題と内容

課題	内容
編集内容が同期されない	編集内容をリアルタイムで同期し、二重編集や情報の不整合を防止する
保存忘れ	自動保存によりデータ消失を防止する
過去の状態へ戻せない	編集履歴を保存し、任意の時点へ復元できるようにする
編集者が分からない	編集者中ユーザーを表示し、共同編集状況を可視化する

実装する機能

機能	内容
ユーザー認証	会員登録、ログイン、ログアウト
ワークスペース管理	作成・更新・削除
メンバー管理	登録済みユーザーの検索・追加・削除
ドキュメント管理	作成・編集・削除
リアルタイム共同編集	WebSocketを利用し複数ユーザー間での編集内容のリアルタイム編集
編集者表示	ドキュメントを開いている・編集しているユーザーをアイコン等で表示
自動保存	一定間隔・一定操作ごとにドキュメント内容を自動的に保存 Redis経由でPostgreSQLへ保存
編集履歴	バージョン管理・復元
セキュリティ	JWT・RateLimit・CSRF・XSS対策
API	REST API・WebSocket API

技術スタック

分類	技術
フロントエンド	React、TypeScript、Vite、Tailwind CSS
エディタ	Monaco Editor
バックエンド	Go、Echo
ORM	GORM
データベース	PostgreSQL
キャッシュ	Redis
認証	JWT
リアルタイム通信	Gorilla WebSocket
API	REST API、WebSocket
インフラ	Docker、Docker Compose
テスト	Go testing、Vitest
CI/CD	GitHub Actions

ユーザー

ゲスト

機能	実装内容	理由	完了条件
会員登録	Email、表示名、パスワードを入力し、アカウントを作成する	NoteHubを利用できるようにするため	正しい入力内容でアカウントが作成される
ログイン	Email、パスワードで認証し、ログインする	登録済みユーザー本人としてサービスを利用するため	正しい認証情報でログインできる

ログインユーザー

ログインした時点では、ホスト・メンバーの役割は持たない

機能	実装内容	なぜ必要？	完了条件
ログアウト	RefreshTokenを無効化し、認証Cookieを削除する	安全にサービスの利用を終了するため	AccessToken・RefreshTokenが無効化される
ワークスペース作成	ワークスペース名を入力し、新しいワークスペースを作成する	共同編集を行う作業環境を作成するため	ワークスペースが作成され、作成者がホストになる

ホスト

ワークスペースを作成したユーザー

機能	実装内容	なぜ必要？	完了条件
ワークスペース編集	ワークスペース名を変更する	管理しやすくするため	ワークスペース情報が更新される
ワークスペース削除	ワークスペースを削除する	不要になったワークスペースを整理するため	ワークスペースが削除される

機能	実装内容	なぜ必要？	完了条件
メンバー追加	登録済みユーザーを検索し、選択してワークスペースへ追加する	メンバーを追加するため	対象ユーザーがワークスペースのメンバーとして登録される
メンバー削除	ワークスペースからメンバーを削除する	アクセス権限を管理するため	対象ユーザーがワークスペースへアクセスできなくなる

メンバー

ホストによってワークスペースへ追加されたユーザー

機能	実装内容	なぜ必要？	完了条件
ドキュメント作成	タイトル・本文を入力しドキュメントを作成する	編集対象を作成するため	ドキュメントが作成される
ドキュメント編集	Monaco Editorでドキュメントを編集する	ドキュメント内容を更新するため	編集内容がリアルタイムで反映される
ドキュメント削除	ドキュメントを削除する	不要なドキュメントを整理するため	ドキュメントが削除される
リアルタイム共同編集	編集内容を他のメンバーへリアルタイムで同期する	複数人で同時に共同編集を行うため	編集内容がリアルタイムで同期される
編集履歴閲覧	保存された編集履歴を一覧表示する	過去の変更内容を確認するため	バージョン一覧が表示される
バージョン復元	任意の編集履歴を現在のドキュメントへ復元する	誤編集を元に戻せるようにするため	選択したバージョンへ復元される

実装手順

機能と実装順	実装内容	なぜ必要？	完了条件
会員登録	名前、メールアドレス、パスワードを入力してアカウントを作成する	NoteHubを利用できるようにするため	正しい入力内容でアカウントが作成される
ログイン	メールアドレス、パスワードで認証する	登録済みユーザー本人として各機能を利用するため	正しい認証情報でログインできる
ログアウト	Refresh Tokenを無効化し、認証Cookieを削除する	共用端末でも安全にログイン状態を終了するため	ログアウト後、同じRefresh TokenからAccess Tokenを再発行できない
RateLimit	会員登録、ログイン、Refresh Token再発行、ワークスペース作成、ドキュメント作成、ドキュメント更新など連続実行による影響が大きいAPIに回数制限を設定する	パスワード総当たり攻撃や大量リクエストによる負荷を防ぐため	制限回数以内は処理され、上限を超えた場合は429 Too Many Requestsを返す
CSRF対策	Refresh TokenをCookieで送信し、Token再発行APIとログアウトAPIでCSRF CookieとX-CSRF-Tokenヘッダーを照合する	他サイトからCookieを悪用されることを防ぐため	正しいCSRF Tokenがある場合だけ処理を実行する

機能と実装順	実装内容	なぜ必要？	完了条件
暫定トップページ作成	ログイン後の確認画面として、ユーザー名、メールアドレス、ログアウトボタンを表示する	認証機能を実装後すぐに動作確認できるようにするため	ログイン成功後にトップページが表示され、未認証時はログイン画面へ遷移する
React Router	公開画面、認証必須画面をルーティングする	権限のない画面へのアクセスを防ぐため	認証が必要な画面は未認証時にログイン画面へ遷移する
AuthContext	ログイン状態、ユーザー情報、認証確認中の状態をReact全体で共有する	各画面で認証状態を統一して管理するため	ページを再読み込みしても認証状態が維持される
ワークスペース作成	ワークスペースを登録し、作成者をホストとして登録する	共同編集を開始するため	ワークスペースが作成され、作成者がホストになる
ユーザー検索	Emailから登録済みユーザーを検索する	追加するユーザーを選択できるようにするため	検索条件に一致する登録済みユーザーが表示される
メンバー追加	選択したユーザーをWorkspaceMemberへMemberとして登録する	共同編集を行うユーザーを追加するため	対象ユーザーがワークスペースのメンバーとして登録される
メンバー一覧取得	ワークスペースに所属するホストとメンバーを取得する	現在の所属ユーザーを確認するため	ホストとメンバーが一覧表示される
メンバー削除	対象ユーザーをWorkspaceMemberから削除する	不要なアクセス権限を解除するため	対象ユーザーがワークスペースへアクセスできなくなる
ドキュメント作成	タイトルと本文を入力してドキュメントを作成する	編集対象を作成するため	ドキュメントが作成される
ドキュメント取得	ドキュメントを取得し、Monaco Editorへ表示する	編集できるようにするため	ドキュメントが表示される
WebSocket接続	ドキュメントを開いたタイミングでWebSocketへ接続する	リアルタイム共同編集を開始するため	WebSocket接続が確立される
編集同期	編集内容を接続中のメンバーへ送信する	全員が最新状態で編集できるようにするため	編集内容がリアルタイムで同期される
自動保存	編集停止後60秒でRedisへ最新状態を保持し、バックグラウンド処理でPostgreSQLへ保存	編集内容の消失を防ぐため	最新の編集内容が保存される
編集履歴保存	編集履歴を保存する	過去の状態へ戻せるようにするため	編集履歴が保存される
バージョン一覧・詳細	保存されたバージョンの本文、更新者、更新日時を取得して表示する	復元する内容を確認して選択できるようにするため	対象ドキュメントの保存済みバージョンが一覧表示され、選択した内容を確認できる
復元前履歴保存	バージョン復元を実行する直前に、現在のドキュメント本文をDocumentVersionへ保存する	復元後でも復元前の状態へ戻せるようにするため	復元前の本文、実行者、保存日時が新しい履歴として保存される
バージョン復元	任意の編集履歴を現在のドキュメントへ反映する	誤編集を元に戻せるようにするため	選択したバージョンへ復元される

機能と実装順	実装内容	なぜ必要？	完了条件
復元履歴保存	復元後の内容を復元操作によって作成された新しい DocumentVersionとして保存する	どの時点の内容を誰が復元したか追跡できるようにするため	復元元バージョン、復元実行者、復元日時を確認できる履歴が保存される
復元元バージョン、復元実行者、復元日時を確認できる履歴が保存される	復元後のドキュメント内容を WebSocket で接続中のユーザーへ同期する	全ユーザーが同じ内容を編集できるようにするため	同じドキュメントを開いている全ユーザーの画面が復元後の内容へ更新される

React Router

項目	内容
実装内容	React Routerを利用して画面遷移を管理する
公開画面	/login、/register
認証必須画面	/workspace、/workspace/:workspaceId、/workspace/:workspaceId/document/:documentId
ホスト専用画面	/workspace/:workspaceId/settings
認証制御	未認証ユーザーが認証必須画面へアクセスした場合は、ログイン画面へ遷移する
権限制御	ホスト権限が必要な画面は、ホスト以外のアクセスを拒否する
ログイン済み制御	ログイン済みユーザーがログイン画面または会員登録画面へアクセスした場合は、ワークスペース一覧画面へ遷移する
完了条件	URLごとに対応する画面が表示され、認証状態と権限に応じたアクセス制御が行われる

AuthContext

項目	内容
実装内容	ログイン状態を React 全体で共有する
管理情報	ログインユーザー情報、認証状態、認証確認中状態
提供機能	ログイン、ログアウト、現在のユーザー情報取得、認証状態更新
初回処理	アプリ起動時に現在のログインユーザー情報を取得する
ページ更新時	ページを再読み込みした場合も、認証状態を再確認する
ログアウト時	認証情報を初期化し、ログイン画面へ遷移する
完了条件	全画面で同じ認証状態が共有され、ページ再読み込み後もログイン状態が維持される

ワークスペース作成

項目	内容
実装内容	ワークスペース名を入力し、新しいワークスペースを作成する
登録内容	ワークスペース情報、ワークスペースメンバー情報
自動処理	作成者をホストとして登録する
利用権限	ログインユーザー
完了条件	ワークスペースが作成され、作成者がホストとして登録される

ユーザー検索

項目	内容
実装内容	登録済みユーザーを検索する
検索条件	メールアドレス
検索方式	完全一致
除外対象	自分自身、すでに参加しているユーザー
利用権限	ホスト
完了条件	検索条件に一致する登録済みユーザーが表示される

メンバー追加

項目	内容
実装内容	ワークスペースへ参加させるユーザーを追加する
入力内容	検索したユーザーを選択する
処理内容	選択したユーザーをワークスペースのメンバーとして登録する
制約	既に参加しているユーザーは追加できない
利用権限	ホストのみ実行できる
完了条件	選択したユーザーがワークスペースへ参加し、ドキュメントを閲覧・編集できる

メンバー一覧取得

項目	内容
実装内容	ワークスペースに所属するホストとメンバーを取得する
取得項目	表示名、メールアドレス、権限、参加日時
表示順	ホスト、メンバー
利用権限	ホスト、メンバー
完了条件	ワークスペースに所属するホストとメンバーが一覧表示される

メンバー削除

項目	内容
実装内容	対象ユーザーをワークスペースメンバーから削除する
制御内容	ホスト自身は削除できない
接続中の処理	対象ユーザーがドキュメントを開いている場合は、WebSocket接続を切断する
権限削除	対象ユーザーのワークスペースおよびドキュメントへのアクセスを拒否する
利用権限	ホスト
完了条件	対象ユーザーがワークスペースへアクセスできなくなる

ドキュメント作成

項目	内容
実装内容	新しいドキュメントを作成する
入力内容	タイトル
処理内容	ワークスペース内へ新しいドキュメントを作成する
制約	ワークスペースのメンバーのみ作成できる
利用権限	ホスト、メンバー
完了条件	新しいドキュメントが作成され、編集画面が表示される

ドキュメント取得

項目	内容
実装内容	ドキュメントの内容を取得する
処理内容	タイトルと本文を取得して編集画面へ表示する
制約	ワークスペースのメンバーのみ閲覧できる
利用権限	ホスト、メンバー
完了条件	ドキュメントの内容が編集画面へ表示される

ドキュメント削除

項目	内容
実装内容	ドキュメントを削除する
処理内容	対象ドキュメントを削除する
制約	ワークスペースのメンバーのみ削除できる
利用権限	ホスト、メンバー
完了条件	ドキュメント一覧から削除される

WebSocket接続

項目	内容
実装内容	ドキュメントを開いたときにWebSocketへ接続する
処理内容	サーバーとのリアルタイム通信を開始する
制約	認証済みかつワークスペースのメンバーのみ接続できる
利用権限	ホスト、メンバー
完了条件	WebSocket接続が確立される

リアルタイム共同編集

項目	内容
実装内容	編集内容をリアルタイムで他の利用者へ反映する
処理内容	編集内容を接続中のメンバーへ配信する
制約	同じドキュメントを開いている利用者のみ同期する
利用権限	ホスト、メンバー

項目	内容
完了条件	編集内容がリアルタイムで同期される

編集者表示

項目	内容
実装内容	現在ドキュメントを開いている利用者を表示する
処理内容	接続中の利用者一覧を表示する
表示内容	表示名
利用権限	ホスト、メンバー
完了条件	編集中の利用者がリアルタイムで表示・更新される

自動保存

項目	内容
実装内容	編集内容を一定時間ごとに自動保存する
処理内容	最新のドキュメント内容を保存する
保存タイミング	編集停止後60秒、または一定間隔
利用権限	ホスト、メンバー
完了条件	最新の編集内容が保存される

編集履歴保存

項目	内容
実装内容	ドキュメントの編集履歴を保存する
処理内容	保存時点の本文、更新者、更新日時を履歴として保存する
保存タイミング	自動保存時、復元前、手動保存時
利用権限	ホスト、メンバー
完了条件	編集履歴が保存される

バージョン一覧

項目	内容
実装内容	保存された編集履歴を一覧表示する
表示内容	更新者、更新日時、バージョン番号
処理内容	過去の編集履歴を取得して表示する
利用権限	ホスト、メンバー
完了条件	保存済みの編集履歴が一覧表示される

バージョン詳細

項目	内容
実装内容	選択した編集履歴の内容を表示する
表示内容	タイトル、本文、更新者、更新日時
利用権限	ホスト、メンバー
完了条件	選択した編集履歴の内容を確認できる

バージョン復元

項目	内容
実装内容	選択した編集履歴を現在のドキュメントへ反映する
処理内容	現在の内容を履歴として保存した後、選択した履歴を復元する
利用権限	ホスト、メンバー
完了条件	ドキュメントが選択したバージョンの内容へ更新される

編集競合防止

項目	内容
実装内容	同じドキュメントを複数人で同時に編集できるようにする
処理内容	編集内容を受信した順番に反映し、他の利用者へ配信する
制約	編集内容はサーバーで順番に処理する
利用権限	ホスト、メンバー
完了条件	編集内容の不整合が発生せず、全利用者で同じ内容が表示される

ドキュメント保存状態

項目	内容
実装内容	ドキュメントの状態を定期的に保存する
保存内容	ドキュメント本文、更新者、更新日時
保存タイミング	一定時間ごと、または一定回数の編集ごと
利用目的	サーバー障害時の復旧および履歴管理
完了条件	最新のドキュメント状態が保存される

WebSocket接続管理

項目	内容
実装内容	ドキュメントごとの接続ユーザーを管理する
管理内容	接続ユーザー、接続日時、対象ドキュメント
処理内容	接続・切断時に一覧を更新する
利用目的	編集者表示とリアルタイム配信を管理する
完了条件	接続状況が常に最新の状態になる

WebSocket再接続

項目	内容
実装内容	通信が切断された場合に自動で再接続する
処理内容	一定時間ごとに再接続を試行する
制約	認証済みユーザーのみ再接続できる
利用目的	一時的な通信切断から自動復旧する
完了条件	通信が復旧すると共同編集へ自動復帰する

Redis管理

項目	内容
実装内容	リアルタイム編集に必要な一時データを管理する
保存内容	編集内容、接続情報、自動保存待ちデータ
利用目的	高速なデータ共有とサーバー負荷軽減
完了条件	一時データがRedisで管理される

PostgreSQL保存

項目	内容
実装内容	ドキュメント情報を永続保存する
保存内容	ドキュメント、編集履歴、ワークスペース情報
処理内容	自動保存時に最新内容を反映する
利用目的	データを永続的に保持する
完了条件	最新の編集内容がデータベースへ保存される

自動復旧

項目	内容
実装内容	サーバー再起動後に編集状態を復元する
処理内容	保存済みドキュメントを読み込み、共同編集を再開する
利用目的	サーバー障害時の影響を最小限にする
完了条件	サーバー再起動後も共同編集を継続できる

編集ログ保存

項目	内容
実装内容	編集操作をログとして保存する
保存内容	編集者、編集日時、操作内容
利用目的	障害調査および監査に利用する
完了条件	編集ログが保存される

障害時の復旧

項目	内容
実装内容	保存済みデータから復旧する
処理内容	最新の保存内容を読み込み、編集を再開する
利用目的	データ消失を防止する
完了条件	保存済みデータから復元できる

バックアップ

項目	内容
実装内容	ドキュメントデータを定期的にバックアップする
対象	ドキュメント、編集履歴、ワークスペース情報
実行タイミング	定期実行
利用目的	障害発生時のデータ復旧に利用する
完了条件	定期的にバックアップが作成される

認証・セキュリティ要件

認証方式

項目	内容
認証方法	メールアドレスとパスワードを使用して本人確認を行う
Access Token	ログイン成功後にJWT形式で発行し、認証が必要なAPIの利用者確認に使用する
Access Tokenの有効期限	15分とする
Access Tokenの保存場所	ブラウザのメモリ上に保持し、LocalStorageやSessionStorageには保存しない
Refresh Token	Access Tokenの有効期限が切れた場合に、新しいAccess Tokenを発行するために使用する
Refresh Tokenの保存場所	JavaScriptから読み取れないHttpOnly Cookieへ保存する
認証失敗時	メールアドレスまたはパスワードのどちらかが間違っているか特定できないメッセージを返す
完了条件	正しい認証情報の場合のみログインでき、認証が必要な機能を利用できる

パスワード

項目	内容
実装内容	パスワードを安全な形式へ変換して保存する
保存方法	bcryptを使用してハッシュ化する
保存禁止	平文のパスワードをデータベース、ログ、レスポンスへ保存・出力しない
パスワード要件	8文字以上15文字以下とし、英字と数字を含める
ログイン時	入力されたパスワードと保存済みのハッシュ値を照合する
利用目的	データベースやログが流出した場合でも、元のパスワードが直接漏れないようにする
完了条件	データベースおよびログに平文のパスワードが存在しない

Access Token認証

項目	内容
実装内容	認証が必要なAPIでAccess Tokenを検証する
送信方法	AuthorizationヘッダーへAccess Tokenを設定する
検証内容	署名、有効期限、ユーザーIDなどを確認する
認証エラー	Tokenがない、期限切れ、または改ざんされている場合は処理を拒否する
利用目的	ログイン済みの本人からのリクエストであることを確認する
完了条件	有効なAccess Tokenを持つユーザーだけが認証必須APIを利用できる

Refresh Token管理

項目	内容
実装内容	Access Tokenの再発行に使用するRefresh Tokenを安全に管理する
Cookie属性	HttpOnly、Secure、SameSiteを設定する
データベース保存	Refresh Tokenの平文ではなく、ハッシュ化した値を保存する
有効期限	30分（本番では14日）
ログアウト時	Refresh Tokenを無効化し、認証Cookieを削除する
利用停止時	対象ユーザーのRefresh Tokenを無効化し、再発行を拒否する
完了条件	JavaScriptからRefresh Tokenを読み取ることができず、無効化済みのTokenは利用できない

Refresh Tokenローテーション

項目	内容
実装内容	Access Tokenを再発行するたびに、新しいRefresh Tokenを発行する
旧Tokenの処理	使用したRefresh Tokenを直ちに無効化する
再利用禁止	使用済みのRefresh Tokenから再度Tokenを発行できないようにする
利用目的	盗まれたRefresh Tokenが繰り返し利用されることを防ぐ
完了条件	Token再発行後、旧Refresh Tokenを使用すると処理が拒否される

Refresh Token再利用検知

項目	内容
実装内容	無効化済みのRefresh Tokenが再度使用されたことを検知する
処理内容	同じログインから発行されたRefresh Tokenをすべて無効化する
ユーザー処理	対象ユーザーを強制的にログアウトさせる
ログ記録	ユーザー、日時、接続元などを監査ログへ記録する
利用目的	Refresh Tokenの盗難が疑われる状態でログインを継続させないため
完了条件	再利用を検知した後は、同じログイン状態からTokenを再発行できない

操作権限の確認

項目	内容
実装内容	ワークスペースおよびドキュメントを操作できるユーザーか確認する
APIの確認	対象ワークスペースに所属しているか、各APIで確認する
WebSocketの確認	接続時およびメッセージ受信時に、対象ドキュメントの所属メンバーか確認する
ホスト権限	メンバー追加・削除やワークスペース設定はホストのみ実行できる
権限変更時	メンバーから削除されたユーザーのAPIおよびWebSocket利用を直ちに拒否する
完了条件	所属していないワークスペースやドキュメントを閲覧・編集できない

HTTPS・WSS通信

項目	内容
API通信	本番環境ではHTTPS通信を必須とする
WebSocket通信	WSS通信を必須とし、暗号化されていないWS通信は禁止する
HTTPアクセス	HTTPでアクセスされた場合はHTTPSへ転送する
利用目的	認証情報やドキュメント内容が通信経路上で盗み見られることを防ぐ
完了条件	本番環境のAPIとWebSocketが暗号化された通信でのみ利用できる

WebSocket認証

項目	内容
実装内容	WebSocket接続を開始する前にログイン状態を確認する
認証方法	接続開始時にAccess Tokenを送信し、サーバー側で検証する
検証内容	Tokenの署名、有効期限、ユーザー情報を確認する
権限確認	対象ドキュメントが所属するワークスペースのメンバーか確認する
認証失敗時	未認証、Token期限切れ、権限不足の場合は接続を拒否する
接続中の権限変更	メンバーから削除された場合は対象のWebSocket接続を切断する
完了条件	認証済みかつ対象ドキュメントを利用できるユーザーだけが接続できる

CSRF対策

項目	内容
対象API	Refresh Token再発行、ログアウトなどCookieを使用するAPI
対策方式	Double Submit Cookie方式を使用する
処理内容	CSRF Cookieの値とX-CSRF-Tokenヘッダーの値を照合する
不一致時	値が存在しない、または一致しない場合は処理を拒否する
エラー	403 Forbiddenを返す
利用目的	他のWebサイトから認証Cookieを不正利用されることを防ぐ
完了条件	正しいCSRF Tokenが送信された場合だけ対象APIが実行される

CORS対策

項目	内容
実装内容	APIへのアクセスを許可するフロントエンドの接続元を制限する
許可対象	本番環境のフロントエンドドメイン
禁止事項	すべての接続元を無条件で許可しない
Cookie通信	認証Cookieを使用するため、許可ドメインと認証情報の送信設定を明示する
利用目的	許可していないWebサイトからAPIを利用されることを防ぐ
完了条件	許可されたフロントエンドからの通信だけが受け付けられる

Rate Limit

項目	内容
実装内容	短時間に同じ操作が繰り返された場合にリクエストを制限する
対象	会員登録、ログイン、Token再発行、ユーザー検索、ワークスペース作成、ドキュメント作成・更新
管理単位	未認証時はIPアドレス、認証済みの場合はユーザー単位で管理する
管理方法	Redisを使用したToken Bucket方式を採用する
上限超過時	UseCaseの処理を実行せず、429 Too Many Requestsを返す
ログ記録	制限対象、ユーザーまたはIPアドレス、日時を記録する
利用目的	パスワードの総当たり攻撃、大量登録、過剰な更新処理を防ぐ
完了条件	制限回数以内は処理され、上限を超えたリクエストは拒否される

ログイン試行制限

項目	内容
実装内容	同じメールアドレスまたはIPアドレスからログイン失敗が繰り返された場合に制限する
処理内容	一定回数（5回）以上失敗した場合、一定時間（30秒 本番は1時間）ログイン処理を拒否する
エラーメッセージ	アカウントの存在有無が特定されない共通メッセージを返す
成功時	ログイン成功後は失敗回数をリセットする
ログ記録	制限開始日時、対象、接続元を監査ログへ記録する
利用目的	パスワードの総当たり攻撃を防ぐ
完了条件	設定回数を超えて失敗した場合は、一時的にログインできなくなる

XSS対策

項目	内容
対象	ドキュメント本文、タイトル、ワークスペース名、表示名などのユーザー入力
実装内容	ユーザーが入力した内容を、そのままHTMLとして画面へ表示しない
禁止対象	scriptタグ、iframeタグ、onClickなどのイベント属性、javascript: URL
サニタイズ	HTMLを扱う場合は、安全な要素と属性だけを残し、危険な内容を除去する
フロントエンド	通常の文字列はReactの標準表示を使用し、危険なHTML直接挿入を禁止する
バックエンド	入力値の形式、長さ、許可文字を検証する
利用目的	不正なスクリプトの実行や認証情報の盗難を防ぐ

項目	内容
完了条件	保存した入力内容を表示しても、不正なスクリプトが実行されない

同時接続数制限

項目	内容
実装内容	同じユーザーが同じドキュメントを複数タブで開ける数に上限を設定する
管理単位	同一ユーザー、同一ドキュメント単位で管理する
上限値	最大2接続とする
上限超過時	新しいWebSocket接続を拒否する
エラー表示	接続可能なタブ数の上限を超えたことを画面へ表示する
ログ記録	ユーザー、ドキュメント、接続日時、拒否理由を記録する
利用目的	不要な多重接続によるサーバー負荷や編集者一覧の混乱を防ぐ
完了条件	上限を超える接続は受け付けられず、既存の接続は継続される

入力値検証

項目	内容
実装内容	APIおよびWebSocketで受け取る入力値を検証する
検証対象	必須項目、文字数、形式、識別子、列挙値、本文サイズ
ドキュメント本文	保存可能な最大文字数または最大データサイズを設定する
不正入力時	データベース処理を行わず、入力エラーを返す
クライアント側検証	操作性向上のために行うが、サーバー側でも必ず再検証する
利用目的	不正データの登録や過大な入力による負荷を防ぐ
完了条件	条件を満たさない入力は保存・配信されない

エラー情報の保護

項目	内容
実装内容	利用者へ内部の詳細情報を含まないエラーメッセージを返す
非表示対象	SQL文、サーバー内部パス、Token、パスワード
サーバーログ	調査に必要な詳細情報はサーバー側のログへ記録する
本番環境	デバッグ用の詳細エラーを画面やAPIレスポンスへ出力しない
完了条件	エラー発生時にシステム内部情報や認証情報が利用者へ漏れない

データモデル

テーブル	内容	項目
User	ユーザーアカウント情報	id, email, password_hash, name, status, created_at, updated_at
Workspace	ワークスペース情報	id, name, host_id, created_at, updated_at

テーブル	内容	項目
WorkspaceMember	ワークスペースとユーザーの所属関係・権限管理	id, workspace_id, user_id, role (Host/Member) , joined_at, created_at, updated_at
Document	ドキュメントの現在情報	id, workspace_id, title, content, created_by, updated_by, created_at, updated_at
DocumentVersion	ドキュメントの編集履歴	id, document_id, content, created_by, version_type, source_version_id, created_at
EditSession	リアルタイム編集集中のユーザー接続状態	id, document_id, user_id, connection_id, connected_at, last_active_at, disconnected_at
RefreshSession	ログインセッションとRefreshToken管理	id, user_id, token_hash, family_id, status, expires_at, created_at, rotated_at, revoked_at, last_used_at
LoginAttempt	ログイン失敗回数と一時ロック管理	id, email, ip_address, failed_count, locked_until, last_failed_at, created_at, updated_at
AuditLog	操作の監査記録	id, user_id, action, target_type, target_id, workspace_id, document_id, ip_address, details, created_at
APIRequestLog	API処理ログ	id, request_id, user_id, method, path, status, latency_ms, error_code, created_at
WebSocketLog	WebSocket接続・切断・拒否の記録	id, user_id, document_id, connection_id, event_type, reason, ip_address, created_at

API

API	内容
POST /auth/signup	ユーザー登録
POST /auth/login	ログイン (JWT発行)
POST /auth/logout	ログアウト (AccessToken、RefreshTokenを無効化し、Cookieに保存されている認証情報を削除)
POST /auth/refresh	AccessToken再発行
GET /workspaces	所属ワークスペース一覧取得
POST /workspaces	ワークスペース作成
PATCH /workspaces/:id	ワークスペース更新
DELETE /workspaces/:id	ワークスペース削除
GET /workspaces/:id/members	メンバー一覧取得
GET /users?keyword=:keyword	登録済みユーザー検索
POST /workspaces/:id/members	登録済みユーザーをメンバーとして追加
DELETE /workspaces/:id/members/:userId	メンバー削除

API	内容
GET /workspaces/:id/documents	ドキュメント一覧取得
POST /workspaces/:id/documents	ドキュメント作成
GET /documents/:id	ドキュメント詳細取得
PATCH /documents/:id	ドキュメント更新
DELETE /documents/:id	ドキュメント削除
GET /documents/:id/editors	編集者一覧取得
GET /documents/:id/versions	編集履歴一覧取得
POST /documents/:id/restore	編集履歴復元
WS /ws/documents/:id	リアルタイム共同編集用WebSocket接続

AccessToken

項目	内容
形式	JWT
有効期限	15分
保持場所	クライアントのメモリ上のみ（localStorage／sessionStorageには保存しない）
格納情報	user_id、ワークスペース権限情報、発行日時、有効期限
用途	id, document_id, content, created_by, created_at
EditSession	REST API・WebSocket接続の双方で本人確認に利用

セキュリティ・運用要件

レート制限（Token Bucket方式）

項目	内容
アルゴリズム	Token Bucket方式
実装方式	Redis + Luaスクリプト 処理を途中で中断せず、一つのまとまりとして実行
スクリプト言語	Lua
制限超過時	429 Too Many Requestsを返却

API・イベントごとのバケット設定

項目	管理単位	バケット容量
login	IPアドレス＋アカウント	5トークン（1分あたり1トークン）
signup	IPアドレス	3トークン（1時間あたり）
refresh	ユーザー	10トークン（1分あたり5トークン）
search	ユーザー	20トークン（1分あたり10トークン）
ドキュメント作成、更新	ユーザー	20トークン（1分あたり10トークン）

項目	管理単位	バケット容量
WebSocket編集、接続	WebSocket接続時	30トークン（1秒あたり10トークン）

タブ開きすぎ防止

項目	内容
制限対象	同一ユーザー・同一ドキュメントの同時接続（タブ）数
上限	同一ドキュメントにつき最大3タブ
超過時	新規接続を拒否

同時ログイン端末数制限

項目	内容
管理単位	1つのRefreshTokenを1つのログインセッション
上限値	2（1アカウントあたりの同時ログイン可能な数）
超過時	最も古いセッションを無効化
セッション管理	ユーザーは有効なログインセッション一覧を確認し、任意のセッションを無効化できる

ログイン試行制限

項目	内容
目的	ブルートフォース攻撃対策
制限方式	一定時間内の連続ログイン失敗回数を管理
上限値	5回失敗で一時ロック
ロック内容	15分間ログインを拒否
記録単位	アカウント単位およびIPアドレス単位で管理
通知	ロック発生時は監査ログへ記録する

WebSocket編集イベント制限

項目	内容
目的	大量送信によるサーバー負荷を防止
制限方式	接続ごとにToken Bucket方式で送信可能イベント数を制限
バケット容量	接続ごとに最大30トークン
トークン補充	1秒あたり10トークンを補充する
超過時	リクエストを破棄

通信セキュリティ

項目	内容
API通信	HTTPS必須
WebSocket通信	WSSを必須、暗号化されていないWebSocket通信は禁止

項目	内容
Cookie属性	Secure、HttpOnly、SameSiteを設定
CORS	フロントエンドのドメインのみ許可

リフレッシュトークン

項目	内容
保存場所	HttpOnly Cookie
DB保存	ハッシュ化した値のみ保存し、平文は保存しない
有効期限	30分（本番想定ならAccessTokenより長い期間 14日）
ローテーション	リフレッシュ時に新しいRefreshTokenを発行し、旧トークンを無効化する
再利用検知	無効化済みトークンの利用を検知した場合はセッションを強制ログアウトし、監査ログへ記録する
CSRF対策	Cookieを利用するAPIはDouble Submit Cookie方式を採用する

ログ要件

API処理ログ

項目	内容
ログ項目	request_id、method、path、status、latency_ms、user_id、error_code

監査ログ

項目	内容
記録対象	ログイン、ログアウト、RefreshToken再利用検知、ログイン試行制限発動、権限変更、ワークスペース作成・削除、ドキュメント削除、バッチ処理実行

Rate Limit・WebSocketログ

項目	内容
記録対象	レート制限超過、WebSocket強制切断、同時接続数超過による接続拒否

XSS対策

項目	内容
対象	ドキュメント本文、タイトルなどユーザーが入力した内容
対策	表示時にHTMLから安全な内容だけを表示、scriptタグ、iframeタグ、イベント属性（onClick等）、javascript: URLなどの危険な要素を除去する

インフラ・デプロイ要件

項目	内容
フロントエンド	React、TypeScript、Vite、Tailwind CSSで実装し、Dockerコンテナとしてデプロイする
バックエンド	Go（Echo）によるREST API・WebSocketサーバーをDockerコンテナ
DB	PostgreSQL（GORMを利用）
Redis（Pub/Sub用）	WebSocketのPub/Subおよび編集者の管理に利用する
Redis（自動保存用）	編集中心ドキュメントのワーキングキャッシュとして利用し、60秒ごとにPostgreSQLへ永続化する
コンテナ管理	Docker Composeでフロントエンド、バックエンド、PostgreSQL、Redisを一括起動する
CI/CD	GitHub Actionsによるビルド、テスト、デプロイを自動化する